

PLP - Primer Parcial - 1^{er} cuatrimestre de 2024

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

El siguiente tipo de datos sirve para representar árboles ternarios:

```
data AT a = NilT | Tri a (AT a) (AT a) (AT a)
```

Definimos el siguiente árbol para los ejemplos:

```
at1 = Tri 1 (Tri 2 NilT NilT NilT) (Tri 3 (Tri 4 NilT NilT NilT) NilT NilT)
      (Tri 5 NilT NilT NilT)
```

a) Dar el tipo y definir la función `foldAT` que implementa el esquema de recursión estructural para el tipo `AT a`. Sólo en este inciso se permite usar recursión explícita.

b) Definir la función `preorder :: AT a -> [a]`, que lista los nodos de un árbol ternario en el orden en que aparecen: primero la raíz, después los nodos del subárbol izquierdo, luego los del medio y finalmente los del derecho.

Por ejemplo: `preorder at1 ~> [1, 2, 3, 4, 5]`.

c) Definir la función `mapAT :: (a -> b) -> AT a -> AT b`, análoga a la función `map` para listas, pero para árboles ternarios.

Por ejemplo: `mapAT (+1) at1 ~> Tri 2 (Tri 3 NilT NilT NilT) (Tri 4 (Tri 5 NilT NilT) NilT NilT) (Tri 6 NilT NilT NilT)`.

d) Definir la función `nivel :: AT a -> Int -> [a]`, que devuelve la lista de nodos del nivel correspondiente del árbol, siendo 0 el nivel de la raíz.

Por ejemplo: `nivel at1 1 ~> [2, 3, 5]`.

Pista: aprovechar la curriificación y utilizar evaluación parcial.

Ejercicio 2 - Demostración de propiedades

Considerar las siguientes definiciones sobre listas y árboles estrictamente binarios¹:

```
data AEB a = Hoja a | Bin (AEB a) a (AEB a)
```

```
const :: a -> b -> a
```

```
{C} const = (\ x -> \ y -> x)
```

```
head :: [a] -> a
```

```
{H} head (x:xs) = x
```

```
tail :: [a] -> [a]
```

```
{T} tail (x:xs) = xs
```

```
length :: [a] -> Int
```

```
{L0} length [] = 0
```

```
{L1} length (x:xs) = 1 + length xs
```

```
null :: [a] -> Bool
```

```
{N0} null [] = True
```

```
{N1} null (x:xs) = False
```

¹Escritas con recursión explícita para facilitar las demostraciones.

```

altura :: AEB a -> Int
{A0} altura (Hoja x) = 1
{A1} altura (Bin i r d) = 1 + max (altura i) (altura d)

esPreRama :: Eq a => AEB a -> [a] -> Bool
{E0} esPreRama (Hoja x) = \xs -> null xs || (xs == [x])
{E1} esPreRama (Bin i r d) = \xs -> null xs ||
    (r == head xs && (esPreRama i (tail xs) || esPreRama d (tail xs)))

```

a) Asumiendo Eq a, demostrar la siguiente propiedad:

$\forall t :: AEB\ a. \forall xs :: [a]. esPreRama\ t\ xs \Rightarrow length\ xs \leq altura\ t$

Se recomienda hacer inducción en el árbol, utilizando extensionalidad de booleanos y listas cuando sea necesario. Se permite definir macros (i.e., poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio escribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes. Recordar también que los $=$ de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos, así como también que $\forall t :: AEB\ a. altura\ t \geq 0$.

b) Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$((P \Rightarrow Q) \wedge (Q \Rightarrow R)) \Rightarrow \neg R \Rightarrow \neg P$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda simplemente tipado para modelar **Diccionarios**. Para eso se extienden los tipos y expresiones de la siguiente manera:

$\tau ::= \dots \mid Dicc(\tau, \tau)$
 $M ::= \dots \mid Vacío_{\sigma, \tau} \mid definir(M, M, M) \mid def?(M, M) \mid obtener(M, M)$

- $Dicc(\sigma, \tau)$ es el tipo de los diccionarios con claves de tipo σ y valores de tipo τ .
- $Vacío_{\sigma, \tau}$ es un diccionario vacío con claves de tipo σ y valores de tipo τ .
- $definir(M, N, O)$ define el valor O en el diccionario M para la clave N .
- $def?(M, N)$ indica si la clave N fue definida en el diccionario M .
- $obtener(M, N)$ da el valor asociado a la clave N en el diccionario M (se espera que el diccionario tenga definida la clave y, en caso contrario, la expresión puede tipar, pero no se obtendrá un valor).

a. Introducir las reglas de tipado para la extensión propuesta.

b. Definir el conjunto de valores y las nuevas reglas de semántica operacional a pequeños pasos. Suponer que el tipo de las claves cuenta con el operador $==$ (es decir, el cálculo está extendido con un operador de comparación para los tipos que se usen como claves). Para el caso de $obtener(M, N)$, se espera que la clave N esté definida en el diccionario M . Si no lo está, puede colgarse o terminar en una expresión de error (una forma normal que no es un valor). **No es necesario escribir las reglas de congruencia**, sino que basta con indicar cuántas son.

c. Mostrar paso por paso cómo reduce la expresión:

$(\lambda d: Dicc(Nat, Bool).if\ def?(d, 0)\ then\ obtener(d, 0)\ else\ False)\ definir(Vacío_{Nat, Bool}, 0, True)$

Suponer que $zero == zero \rightarrow True$.

PLP - Recuperatorio del Primer Parcial - 1^{er} cuatrimestre de 2024

#Orden	Nro. Libreta	Apellido(s)	Nombre(s)
81		Fialkowski	Valentín.

Corregido por	Nota E1	Nota E2	Nota E3	Nota Final
PERLA	B-	B-	B-	A

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

En este ejercicio vamos a modelar lógica proposicional en Haskell, de modo de poder construir fórmulas proposicionales y evaluarlas bajo distintas valuaciones.

```
data Prop = Var String | No Prop | Y Prop Prop | O Prop Prop | Imp Prop Prop
```

```
type Valuación = String -> Bool
```

Por ejemplo, la expresión: $Y (Var 'P') (No (Imp (Var 'Q') (Var 'R')))$ representa la proposición $P \wedge \neg(Q \Rightarrow R)$.

Las valuaciones se representan como funciones que a cada variable proposicional le asignan un valor booleano. Por ejemplo la valuación $\lambda x \rightarrow x == 'P'$ le asigna el valor verdadero a la variable P y falso a todas las otras variables proposicionales.

- Dar el tipo y definir las funciones `foldProp` y `recProp`, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo `Prop`. Solo en este inciso se permite usar recursión explícita.
- Definir la función `variables :: Prop -> [String]`, que dada una fórmula devuelve la lista con todas sus variables proposicionales en algún orden, sin elementos repetidos.
Por ejemplo: `variables (O (Var 'P') (No (Y (Var 'Q') (Var 'P'))))` debería devolver la lista `['P', 'Q']` o la lista `['Q', 'P']`.
- Definir la función `evaluar :: Valuación -> Prop -> Bool`, que indica si una fórmula es verdadera o falsa para una valuación dada.
- Definir la función `estáEnFNN :: Prop -> Bool`, que indica si una fórmula está en Forma Normal Negada. Es decir, si no tiene implicaciones y la negación se aplica únicamente a variables y no a proposiciones más complejas.

Por ejemplo: $Y (Var 'P') (No (Imp (Var 'Q') (Var 'R')))$ no está en FNN, y en cambio $Y (Var 'P') (Y (Var 'Q') (No (Var 'R')))$ sí lo está.

Ejercicio 2 - Demostración e inferencia

Considerar las siguientes definiciones sobre árboles con información en las hojas¹:

```
data AIH a = Hoja a | Bin (AIH a) (AIH a)
    der :: AIH a -> AIH a
    {D} der (Bin i d) = d
    esHoja :: AIH a -> Bool
    {E0} esHoja (Hoja x) = True
    {E1} esHoja (Bin i d) = False
    mismaEstructura :: AIH a -> AIH a -> Bool
    {M0} mismaEstructura (Hoja x) = esHoja
    {M1} mismaEstructura (Bin i d) = \t -> not (esHoja t) &&
        mismaEstructura i (izq t) && mismaEstructura d (der t)
    izq :: AIH a -> AIH a
    {I} izq (Bin i d) = i
```

a) Demostrar la siguiente propiedad:

$\forall t :: AIH a. \forall u :: AIH a. mismaEstructura t u = mismaEstructura u t$

Se recomienda hacer inducción en el primer árbol, utilizando extensionalidad en el segundo. Se permite definir macros (poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio reescribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes y escribir los que correspondan al plantear la propiedad como predicado unario. Recordar también que los $=$ de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos.

b) Usar el algoritmo W para inferir juicios de tipado válidos para las siguientes expresiones, o indicar por qué no es posible (recordar que en inferencia **no está permitido** renombrar variables):

i) $(\lambda x. x (\lambda x. Succ(x))) (\lambda x. x)$

ii) $\lambda x. if isZero(x) then x else x zero$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda simplemente tipado para modelar **Árboles con información en las hojas**. Para eso se extienden los tipos y expresiones de la siguiente manera:

$\tau ::= \dots \mid AIH(\tau)$

$M ::= \dots \mid Hoja(M) \mid Bin(M, M) \mid case M of Hoja x \rightsquigarrow M; Bin(i, d) \rightsquigarrow M$

- $AIH(\tau)$ es el tipo de los árboles con información en las hojas de tipo τ .
- $Hoja(M)$ es un árbol compuesto por una única hoja con información M .
- $Bin(M_1, M_2)$ es un árbol compuesto por dos subárboles M_1 y M_2 .
- El observador $case M_1 of Hoja x \rightsquigarrow M_2; Bin(i, d) \rightsquigarrow M_3$ permite acceder al valor de un árbol que es hoja (el cual se ligará a la variable x que puede aparecer libre en M_2), y a los dos subárboles de un árbol que no es hoja (los cuales se ligarán a las variables i y d que pueden aparecer libres en M_3).

- Introducir las reglas de tipado para la extensión propuesta.
- Definir el conjunto de valores y las nuevas reglas de semántica operacional a pequeños pasos, tanto de congruencia como de cómputo.
- Mostrar paso por paso cómo reduce la expresión:
 $case (\lambda n: Nat. Hoja(n)) Succ(zero) of Hoja x \rightsquigarrow Succ(Pred(x)); Bin(i, d) \rightsquigarrow zero$
- Definir como macro la función $esHoja_\tau$, que toma un $AIH(\tau)$ y devuelve un booleano que indica si es una hoja.

¹Escritas con recursión explícita para facilitar las demostraciones.

PLP - Primer Parcial - 2^{do} cuatrimestre de 2024

#Orden	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
			E	E	E	A

(P)
Para promoción!

¡Excelente examen!

Este examen se aprueba obteniendo al menos dos ejercicios bien (B) y uno regular (R), y se promociona con al menos dos ejercicios muy bien (MB) y uno bien (B). Es posible obtener una aprobación condicional con un ejercicio muy bien (MB), uno bien (B) y uno insuficiente (I), pero habiendo entregado algo que contribuya a la solución del ejercicio. Las notas para cada ejercicio son: -, I, R, B, MB, E. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

El siguiente tipo de datos sirve para representar un buffer con historia que permite escribir o leer en cualquiera de sus posiciones (las posiciones tienen tipo Int). El tipo del buffer es paramétrico en el tipo de los contenidos que se pueden guardar en él. Si se escribe dos veces en la misma posición, el nuevo contenido pisa al anterior (por simplicidad). La lectura elimina el contenido leído.¹

`data Buffer a = Empty | Write Int a (Buffer a) | Read Int (Buffer a)`

Definimos el siguiente buffer para los ejemplos:

`buf = Write 1 'a' $ Write 2 'b' $ Write 1 'c' $ Empty`

- Dar el tipo y definir la función `foldBuffer` y `recBuffer`, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo `Buffer a`. Solo en este inciso se permite usar recursión explícita.
- Definir la función `posicionesOcupadas :: Buffer a -> [Int]`, que lista las posiciones ocupadas en un buffer (sin posiciones repetidas).
Por ejemplo: `posicionesOcupadas buf ~ [1, 2]`.
- Definir la función `contenido :: Int -> Buffer a -> Maybe a`, que devuelva el contenido de una posición en un buffer si hay algo en ella, y `Nothing` en caso contrario.
Por ejemplo:
`contenido 1 buf ~ Just 'a'`.
`contenido (-2) buf ~ Nothing`.
`contenido 1 (Read 1 buf) ~ Nothing`.
- Definir la función `puedeCompletarLecturas :: Buffer a -> Bool`, que indique si todas las lecturas pueden completarse exitosamente (es decir, si cada vez que se intenta leer una posición, hay algo escrito en ella).
Por ejemplo:
`puedeCompletarLecturas (Read 1 Empty) ~ False`.
`puedeCompletarLecturas (Read 1 buf) ~ True`.
`puedeCompletarLecturas (Read 1 $ Read 1 buf) ~ False`.
- Definir la función `deshacer :: Buffer a -> Int -> Buffer a`, que dados un buffer y un número natural n (es decir, un Int que por contexto de uso no es negativo), deshaga las últimas n operaciones del buffer, sacando los n constructores más externos para obtener un buffer como el original antes de realizar dichas operaciones. Si se realizaron menos de n operaciones, el resultado debe quedar vacío.
Por ejemplo: `deshacer 2 buf ~ Write 1 'c' Empty`.

Pista: aprovechar la curificación y utilizar evaluación parcial.

¹Por ejemplo, los buffers `b1 = Write 2 True Empty`, `b2 = Write 2 True (Write 2 False Empty)` y `b3 = Read 1 (Write 2 True (Write 1 False Empty))` tienen todos el valor `True` en la posición 2 y nada en las demás, aunque tienen distinta historia y podrían distinguirse mirando el historial de operaciones realizadas.

Ejercicio 2 - Demostración de propiedades

Considerar las siguientes definiciones²:

```
data AB a = Nil | Bin (AB a) a (AB a)

const :: a -> b -> a
{C} const = (\ x -> \ y -> x)

altura :: AB a -> Int
{A0} altura Nil = 0
{A1} altura (Bin i r d) = 1 + max (altura i) (altura d)

zipAB :: AB a -> AB b -> AB (a,b)
{Z0} zipAB Nil = const Nil
{Z1} zipAB (Bin i r d) = \t -> case t of
    Nil -> Nil
    Bin i' r' d' -> Bin (zipAB i i') (r,r') (zipAB d d')
```

a) Demostrar la siguiente propiedad:

$$\forall t :: AB\ a. \forall u :: AB\ a. altura\ t \geq altura\ (zipAB\ t\ u)$$

Se recomienda hacer inducción en un árbol, utilizando extensionalidad para el otro cuando sea necesario. Se permite definir macros (i.e., poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio escribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes. Recordar también que los $=$ de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos, así como también:

$$\{LEMA\} \forall t :: AB\ a. altura\ t \geq 0$$

b) Demostrar el siguiente teorema usando deducción natural, sin utilizar principios clásicos:

$$((\rho \wedge \sigma) \vee (\rho \wedge \tau)) \Rightarrow (\sigma \wedge \rho) \vee \tau$$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda simplemente tipado para modelar **árboles ternarios**. Para eso se extienden los tipos y expresiones de la siguiente manera:

```
 $\tau ::= \dots \mid AT(\tau)$ 
 $M ::= \dots \mid TNil_{\tau} \mid Tern(M, M, M, M) \mid foldAT\ M \triangleright TNil \rightsquigarrow M; Tern(x, ri, rm, rd) \rightsquigarrow M$ 
```

- $AT(\tau)$ es el tipo de los árboles ternarios con nodos de tipo τ .
- $TNil_{\tau}$ es un árbol ternario vacío que admite valores de tipo τ .
- $Tern(M_1, M_2, M_3, M_4)$ es un árbol ternario con raíz M_1 e hijos M_2, M_3 y M_4 .
- $foldAT\ M_1 \triangleright TNil \rightsquigarrow M_2; Tern(x, ri, rm, rd) \rightsquigarrow M_3$ es el esquema de recursión estructural para árboles ternarios, donde las variables x, ri, rm y rd se ligarán a la raíz y los resultados de la recursión sobre los hijos izquierdo, medio y derecho del árbol M_1 respectivamente.

a) Introducir las reglas de tipado para la extensión propuesta.

b) Definir el conjunto de valores y las nuevas reglas de reducción en un paso.

c) Mostrar paso por paso cómo reduce la expresión:

$(\lambda t: AT(Nat). foldAT\ t \triangleright TNil \rightsquigarrow False; Tern(x, ri, rm, rd) \rightsquigarrow isZero(Pred(x)))\ Tern(1, TNil_{Nat}, TNil_{Nat}, TNil_{Nat})$

d) Definir como macro la función $mapAT_{\sigma, \tau}$, que toma un $AT(\sigma)$ y una función de σ en τ , y aplica dicha función a todos los elementos del árbol.

²Escritas con recursión explícita para facilitar las demostraciones.

PLP - Primer Recuperatorio - 2^{do} cuatrimestre de 2024

#Orden	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final

Este examen se aprueba obteniendo al menos dos ejercicios bien menos (B-) y uno regular (R). Las notas para cada ejercicio son: -, I, R, B-, B. Entregar cada ejercicio en hojas separadas. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

En este ejercicio no se permite utilizar recursión explícita, a menos que se indique lo contrario.

El siguiente tipo de datos sirve para representar *operadores* que realizan operaciones combinadas sobre números enteros. Por simplicidad, modelaremos solamente las sumas y divisiones enteras.

`data Operador = Sumar Int | DividirPor Int | Secuencia [Operador]`

`Sumar n` representa la operación que suma `n` a un número entero. `DividirPor n` representa la operación que divide a un entero por `n` (descartando el resto). `Secuencia ops` representa la composición a izquierda de todas las operaciones en `ops`. En otras palabras, representa la operación de aplicar a un número todas las operaciones en `ops` de izquierda a derecha, siendo resultado de cada operación la entrada de la siguiente. Por ejemplo: `Secuencia [Sumar 5, DividirPor 2]` representa la operación que, dado un entero, le suma 5, y al resultado lo divide por 2.

- Dar el tipo y definir la función `foldOperador`, el esquema de recursión estructural para el tipo `Operador`. Sólo en este inciso se permite usar recursión explícita.
- Definir la función `falla :: Operador -> Bool`, que indica si un operador contiene una división por 0 entre sus operaciones. Por ejemplo:
`falla (Secuencia [Sumar 5, DividirPor 2]) ~ False.`
`falla (Secuencia [Sumar 5, DividirPor 0]) ~ True.`
- Definir la función `aplanar :: Operador -> Operador`, que mapea un operador a uno equivalente pero de un solo nivel (si es una suma o división lo deja igual, si es una secuencia devuelve una secuencia con todas las sumas y divisiones en el orden en el que se realizan, sin agruparlas en subsecuencias). Por ejemplo:
`aplanar (Sumar 1) ~ Sumar 1.`
`aplanar (Secuencia [Sumar 1, Secuencia [DividirPor 3, Sumar 2]]) ~ Secuencia [Sumar 1, DividirPor 3, Sumar 2].`
- Definir la función `componerTodas :: [a->a] -> (a->a)` que, dada una lista de funciones, devuelve el resultado de componerlas todas a izquierda (si la lista es vacía, devuelve la identidad).
Es decir: `componerTodas [f1,f2,...,fN] = ((f1 . f2) . ...fN)`
- Definir la función `aplicar :: Operador -> Int -> Maybe Int`, que devuelve el resultado de aplicar el `Operador` a un número, o `Nothing` en caso de falla. Por ejemplo:
`aplicar (Secuencia [Sumar 5, DividirPor 2]) 2 ~ Just 3.`
`aplicar (Secuencia [Sumar 5, DividirPor 0]) 2 ~ Nothing.`

Pista 1: hacer una función auxiliar que aplique el operador a un entero suponiendo que no falla.

Pista 2: aprovechar la curriificación y utilizar evaluación parcial.

Ejercicio 2 - Demostración de propiedades

Considerar las siguientes definiciones¹:

```

const :: a -> b -> a
{C} const = (\ x -> \ y -> x)

length :: [a] -> Int
{L0} length [] = 0
{L1} length (x:xs) = 1 + length xs

head :: [a] -> a
{H} head (x:xs) = x

tail :: [a] -> [a]
{T} tail (x:xs) = xs

null :: [a] -> Bool
{N0} null [] = True
{N1} null (x:xs) = False

zip :: [a] -> [b] -> [(a,b)]
{Z0} zip [] = const []
{Z1} zip (x:xs) = \ys -> if null ys then [] else (x, head ys):zip xs (tail ys)

```

a) Demostrar la siguiente propiedad:

$$\forall xs :: [a] . \forall ys :: [a] . \text{length} (\text{zip } xs \text{ } ys) = \min (\text{length } xs) (\text{length } ys)$$

Se recomienda hacer inducción en una de las listas, utilizando extensionalidad para la otra cuando sea necesario. Se permite definir macros (i.e., poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio escribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes. Recordar también que los $=$ de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos, así como también:

$$\{\text{LEMA}\} \forall xs :: [a] . \min (\text{length } xs) 0 = 0$$

b) Demostrar el siguiente teorema usando deducción natural: $(\tau \Rightarrow (\sigma \wedge \rho)) \vee (\rho \Rightarrow (\sigma \Rightarrow \tau))$.

Se permite utilizar **principios clásicos**. (**Sugerencia:** usar $\tau \vee \neg \tau$).

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el Cálculo Lambda tipado con colas bidireccionales (también conocidas como *deque*). No vamos a definir aquí los observadores (ya que están en la guía), sino que vamos a enfocarnos en los constructores y el esquema de recursión primitiva.

Se extenderán los tipos y términos de la siguiente manera:

$$\tau ::= \dots \mid \text{Cola}_\tau \quad M ::= \dots \mid \langle \rangle_\tau \mid M \bullet M \mid \text{recr } M \triangleright \langle \rangle \rightsquigarrow M; r, c \bullet x \rightsquigarrow M$$

donde $\langle \rangle_\tau$ es la cola vacía en la que se pueden encolar elementos de tipo τ ; $M_1 \bullet M_2$ representa el agregado del elemento M_2 al **final** de la cola M_1 ; y el esquema de recursión primitiva $\text{recr } M_1 \triangleright \langle \rangle \rightsquigarrow M_2; r, c \bullet x \rightsquigarrow M_3$ permite operar con la cola en sentido contrario, accediendo al último elemento encolado (cuyo valor se ligará a la variable x en M_3), al resto de la cola (que se ligará a la variable c en el mismo subtérmino) y al resultado de la recursión sobre el resto de la cola (que se ligará a la variable r).

a) Introducir las reglas de tipado para la extensión propuesta.

b) Demostrar la validez del siguiente juicio de tipado:

$$\emptyset \vdash \lambda c: \text{Cola}_{\text{Nat}}. \text{recr } c \triangleright \langle \rangle \rightsquigarrow \langle \rangle_{\text{Bool}}; r, y \bullet x \rightsquigarrow r \bullet \text{True}: \text{Cola}_{\text{Nat}} \rightarrow \text{Cola}_{\text{Bool}}$$

c) Definir el conjunto de valores y las nuevas reglas de semántica.

d) Mostrar paso por paso cómo reduce la expresión (pueden definir macros para no escribir muchas veces lo mismo.):

$$\text{recr } \langle \rangle_{\text{Nat}} \bullet \text{zero} \bullet \underline{1} \triangleright \langle \rangle \rightsquigarrow \langle \rangle_{\text{Nat}}; r, c \bullet x \rightsquigarrow \text{if isZero}(x) \text{ then } c \text{ else } r \bullet x$$

¹Escritas con recursión explícita para facilitar las demostraciones.

PLP - Primer Recuperatorio - 1^{er} cuatrimestre de 2025

#Orden	Turno	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
--------	-------	---------	-------------------	-----	-----	-----	------------

Las notas para cada ejercicio son: -, I, R, B, MB. Se promueve obteniendo al menos dos ejercicios B y uno R, y se promociona con al menos dos ejercicios MB y uno B. Es posible obtener una aprobación condicional con un ejercicio MB, uno B y uno I. Esto requiere entregar algo que contribuya a la solución del ejercicio. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

Se define el tipo de datos `LineaProd`, que representa líneas de producción de una fábrica. Una línea de producción se inicia con una lista de materiales, donde cada material se representa con un string. Tras esta primera etapa, se puede agregar un material a la línea, o se la puede unir con otra línea de producción.

`data LineaProd = Materiales [String] | Agregar String LineaProd | Unir LineaProd LineaProd`
Definimos las siguientes líneas para los ejemplos:

```
11 = Unir
    (Materiales ["acero", "cobre"])
    (Unir (Agregar "madera" (Materiales ["madera"])) (Materiales ["mercurio"]))
12 = Unir
    (Materiales ["acero", "cobre"])
    (Unir
        (Agregar "madera" (Materiales ["aluminio"]))
        (Agregar "aluminio" (Materiales ["plástico"])))
13 = Unir
    (Materiales ["m1"])
    (Unir (Agregar "m3" (Materiales ["m3", "m4"])) (Materiales ["m1", "m2"]))
```

a) Dar los tipos y definir las funciones `foldLineaProd` y `recLineaProd`, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo `LineaProd`. Solo en este inciso se permite usar recursión explícita.

b) Definir la función `materialesUsados :: LineaProd -> [String]`, que devuelve una lista con todos los materiales usados por una línea de producción. La lista devuelta no debe contener materiales repetidos. Por ejemplo:

```
materialesUsados 11 ~ ["acero", "cobre", "madera", "mercurio"]
materialesUsados 12 ~ ["acero", "cobre", "madera", "aluminio", "plástico"]
```

Sugerencia: usar `nub`. → recibe una lista, elimina los repetidos

c) Definir la función `sublineasDisjuntas :: LineaProd -> Bool`, que dada una línea de producción devuelve `True` si y solo si en ninguna etapa de la misma se unen dos líneas que usen al menos un material en común. Por ejemplo:

```
sublineasDisjuntas 11 ~ True
sublineasDisjuntas 12 ~ False, ya que dentro de ella hay dos sublíneas que se unen y tienen el material "aluminio" en común.
sublineasDisjuntas 13 ~ False, ya que 13 por sí misma es la unión de dos sublíneas que tienen a "m1" en común.
```

d) Definir la función `mismaEstructura :: LineaProd -> LineaProd -> Bool`, que dadas dos líneas de producción determina si son iguales sin considerar los materiales presentes en ellas. Por ejemplo:

```
mismaEstructura 11 12 ~ False
mismaEstructura 11 13 ~ True
```

Pista: aprovechar la curriificación y utilizar evaluación parcial.

Ejercicio 2 - Demostración de propiedades

a) Considerar las siguientes definiciones:

```
data AIH a = Hoja a | Bin (AIH a) (AIH a)

(+++) :: [a] -> [a] -> [a]
{++0} [] ++ ys = ys
{++1} (x:xs) ++ ys = x : (xs ++ ys)

reverse :: [a] -> [a]
{R} reverse = foldr (\ x rec -> rec ++ (x:[])) []

foldr :: (a -> b) -> b -> [a] -> b
{F0} foldr f z [] = z
{F1} foldr f z (x:xs) = f x (foldr f z xs)

hojas :: AIH a -> [a]
{H0} hojas (Hoja h) = [h]
{H1} hojas (Bin i d) = hojas i ++ hojas d

espejo :: AIH a -> AIH a
{E0} espejo (Hoja h) = Hoja h
{E1} espejo (Bin i d) = Bin (espejo d) (espejo i)
```

Demostrar la siguiente propiedad:

$$\forall x :: \text{AIH } a . \text{hojas } (\text{espejo } x) = \text{reverse } (\text{hojas } x)$$

Si se necesita un lema relacionado con `reverse` y `(++)`, no es necesario demostrarlo, pero sí enunciarlo, plantearlo como predicado unario y enunciar cada caso que se debería analizar para demostrarlo por inducción estructural, indicando la hipótesis inductiva para el o los caso(s) inductivo(s).

Se permite definir macros (i.e., poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio escribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes. Recordar también que los `=` de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos.

b) Demostrar el siguiente teorema usando Deducción Natural, sin utilizar principios clásicos:

$$(\rho \Rightarrow \tau) \wedge (\sigma \Rightarrow \tau) \Rightarrow (\sigma \vee \rho) \Rightarrow \tau$$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda simplemente tipado para modelar regiones geométricas. Para eso se extienden los tipos y expresiones de la siguiente manera:

$\tau ::= \dots \mid \text{Región}$

$M ::= \dots \mid \text{Rectángulo}(M, M) \mid \text{Unión}(M, M) \mid \text{foldRG } M \triangleright \text{Rectángulo}(x, y) \rightsquigarrow M; \text{Unión}(ri, rd) \rightsquigarrow M$

- Región es el tipo de las regiones del plano formadas por rectángulos y uniones de rectángulos.
- $\text{Rectángulo}(M_1, M_2)$ es un rectángulo de lados enteros no negativos, de ancho dado por M_1 y alto dado por M_2 , cuyo extremo inferior izquierdo se encuentra en el cero del plano de coordenadas.
- $\text{Unión}(M_1, M_2)$ es la región del plano formada por la unión de las regiones M_1 y M_2 .
- $\text{foldRG } M_1 \triangleright \text{Rectángulo}(x, y) \rightsquigarrow M_2; \text{Unión}(ri, rd) \rightsquigarrow M_3$ es el esquema de recursión estructural para regiones, donde las variables x e y se ligarán en M_2 al ancho y alto del rectángulo, mientras que ri y rd se ligarán en M_3 a los resultados de la recursión sobre las regiones unidas.

a) Introducir las reglas de tipado para la extensión propuesta.

b) Definir el conjunto de valores y las nuevas reglas de reducción en un paso.

c) Mostrar paso por paso cómo reduce la expresión:

$\text{foldRG } \text{Unión}((\lambda x : \text{Nat} . \text{Rectángulo}(1, 2)) \text{ zero}, \text{Rectángulo}(3, 4)) \triangleright \text{Rectángulo}(x, y) \rightsquigarrow \text{True};$

$\text{Unión}(ri, rd) \rightsquigarrow \text{if } ri \text{ then } rd \text{ else False}$

d) Definir como macro la función `contieneIPunto`, que toma una Región y dos números naturales x e y , y devuelve `True` cuando el punto (x, y) está incluido en la región o en su borde, y `False` en caso contrario.

Asumir que se cuenta con la función $\leq : \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Bool}$, la cual compara números naturales. Se pueden usar las macros `and` y `or` definidas en la práctica.

PLP - Primer Parcial - 1^{er} cuatrimestre de 2025 - Parte 2

#Orden	Libreta	Apellido y Nombre	Ej1	Ej2	Ej3	Nota Final
		Pérez Natalia	E	E	E	P

(Promocionado)

Las notas para cada ejercicio son: -, I, R, B, MB, E. Este examen se aprueba obteniendo al menos dos ejercicios B y uno R, y se promociona con al menos dos ejercicios MB y uno B. Es posible obtener una aprobación condicional con un ejercicio MB, uno B y uno I. Esto requiere entregar algo que contribuya a la solución del ejercicio. Poner nombre, apellido y número de orden en todas las hojas, y numerarlas. Se puede utilizar todo lo definido en las prácticas y todo lo que se dio en clase, colocando referencias claras. El orden de los ejercicios es arbitrario. Recomendamos leer el parcial completo antes de empezar a resolverlo.

Ejercicio 1 - Programación funcional

Aclaración: en este ejercicio no está permitido utilizar recursión explícita, a menos que se indique lo contrario.

Se define el tipo de datos ABNV a, que representa árboles binarios no vacíos con elementos de tipo a:

`data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a)`

Definimos el siguiente árbol para los ejemplos:

`abnv = Bi (Uni 2 (Hoja 1)) 3 (Bi (Hoja 4) 5 (Uni 2 (Hoja 7)))`

- Definir las funciones `foldABNV` y `recABNV`, que implementan respectivamente los esquemas de recursión estructural y primitiva para el tipo ABNV a. Solo en este inciso se permite usar recursión explícita.
- Definir la función `elemABNV :: Eq a => a -> ABNV a -> Bool`, que indica si un elemento pertenece a un árbol.
Por ejemplo: `elemABNV 7 abnv ~ True`.
- Definir la función `reemplazarUno :: Eq a => a -> a -> ABNV a -> ABNV a` que, dados dos elementos `x` e `y` y un árbol, devuelve un árbol como el original, pero reemplazando la primera aparición de `x` por `y` (la primera desde la raíz yendo de izquierda a derecha, en el orden de `preorder`).
Por ejemplo:
`reemplazarUno 2 5 abnv ~ Bi (Uni 5 (Hoja 1)) 3 (Bi (Hoja 4) 5 (Uni 2 (Hoja 7)))`.
`reemplazarUno 2 5 (Hoja 1) ~ Hoja 1`.
- Definir la función `nivel :: ABNV a -> Int -> [a]`, que liste todos los elementos del nivel indicado, de izquierda a derecha, siendo 0 el nivel de la raíz. Si el nivel no existe (ya sea por ser negativo o por superar la altura del árbol), devolver `[]`.

Por ejemplo:

`nivel abnv 1 ~ [2,5]`.

`nivel abnv 2 ~ [1,4,2]`.

`nivel abnv 4 ~ []`.

Pista: aprovechar la curificación y utilizar evaluación parcial.

Ejercicio 2 - Demostración de propiedades

- Considerar las siguientes definiciones:

```
data AB a = Nil | Bin (AB a) a (AB a)
```

```
elemAB :: Eq a => a -> AB a -> Bool
```

```
{P0} elemAB _ Nil = False
```

```
{P1} elemAB x (Bin i r d) = (x == r) || (elemAB x i) || (elemAB x d)
```

```
mapAB :: (a -> b) -> AB a -> AB b
```

```
{M0} mapAB _ Nil = Nil
```

```
{M1} mapAB f (Bin i r d) = Bin (mapAB f i) (f r) (mapAB f d)
```


Sean dos tipos a y b tales que valen $\text{Eq } a$, $\text{Eq } b$, y por ende también la siguiente propiedad:

$$\{\text{CONGRUENCIA} ==\} \forall x::a. \forall y::a. \forall f::a \rightarrow b. x == y \Rightarrow f x == f y$$

Demostrar la siguiente propiedad:

$$\forall t::AB \ a. \forall x::a. \forall f::a \rightarrow b. (\text{elemAB } x \ t \Rightarrow \text{elemAB } (f \ x) \ (\text{mapAB } f \ t))$$

Se permite definir macros (i.e., poner nombres a expresiones largas para no tener que repetirlas).

No es obligatorio escribir los $\forall$ correspondientes en cada paso, pero es importante recordar que están presentes. Recordar también que los $=$ de las definiciones pueden leerse en ambos sentidos.

Se consideran demostradas todas las propiedades conocidas sobre enteros y booleanos.

Sugerencia: tener en cuenta que $\forall x::\text{Bool}. \forall y::\text{Bool}. \forall z::\text{Bool}. (x \Rightarrow y) \Rightarrow (x \Rightarrow z \vee y)$.

b) Demostrar el siguiente teorema usando Deducción Natural, sin utilizar principios clásicos:

$$\rho \Rightarrow (\sigma \vee (\rho \Rightarrow \tau)) \Rightarrow (\sigma \vee \tau)$$

Ejercicio 3 - Cálculo Lambda Tipado

Se desea extender el cálculo lambda simplemente tipado para modelar **listas ordenadas**. Una lista ordenada se construye de manera similar a una lista común, pero sus observadores se comportan de manera diferente: la cabeza de la lista es siempre el mínimo elemento, y su cola es el resto de los elementos.

Estas listas tienen como precondition que el tipo de sus elementos cuente con una operación de comparación $<$ (no es necesario escribir nada al respecto, suponemos que vale).

Se extienden los tipos y expresiones de la siguiente manera:

$$\tau ::= \dots \mid [\tau]^<$$

$$M ::= \dots \mid []_\tau^< \mid M ::^< M \mid \text{head}^<(M) \mid \text{tail}^<(M)$$

donde:

- $[\tau]^<$ es el tipo de las listas ordenadas con elementos de tipo τ .
- $[]_\tau^<$ es una lista ordenada vacía que admite valores de tipo τ .
- $M_1 ::^< M_2$ es la lista resultante de agregar el elemento M_1 a la lista M_2 ¹.
- $\text{head}^<(M)$ denota el mínimo elemento de la lista M .
- $\text{tail}^<(M)$ es la lista ordenada con todos los elementos de M excepto el mínimo (si M tiene más de una aparición de su mínimo elemento, se elimina una de ellas).

a) Introducir las reglas de tipado para la extensión propuesta.

b) Definir el conjunto de valores y las nuevas reglas de reducción en un paso. Tener en cuenta que una lista vacía no tiene cabeza ni cola.

Pista: puede ser necesario mirar más de un nivel de un término para saber a qué reduce.

c) Mostrar paso por paso cómo reduce la expresión:

$$\text{head}^<((\lambda x: \text{Nat}. 2)\text{zero}) ::^< 1 ::^< []_{\text{Nat}}^<$$

Se pueden considerar dadas las siguientes reglas:

$$\frac{M_1 \rightarrow M'}{M_1 < M_2 \rightarrow M' < M_2} (E-<_1) \quad \frac{M_2 \rightarrow M'}{V < M_2 \rightarrow V < M'} (E-<_2)$$

$$\frac{}{V < \text{zero} \rightarrow \text{False}} (E-<_F) \quad \frac{}{\text{zero} < \text{Succ}(V) \rightarrow \text{True}} (E-<_T)$$

$$\frac{}{\text{Succ}(V_1) < \text{Succ}(V_2) \rightarrow V_1 < V_2} (E-<_S)$$

¹Notar que M_1 no necesariamente es la cabeza de la lista, y que $::^<$ es un constructor, no es responsable de ordenar los elementos en la lista. El orden aparece al observarla. Por ejemplo, $1 ::^< 2 ::^< []_{\text{Nat}}^<$ y $2 ::^< 1 ::^< []_{\text{Nat}}^<$ son dos valores de tipo $[\text{Nat}]^<$ que tienen la misma cabeza y la misma cola.

Esta parte del examen se aprueba respondiendo correctamente al menos el 75% de las preguntas. Es necesario aprobar esta parte para acceder a la corrección de la segunda parte. Responder las preguntas en los espacios indicados.

Pregunta 1: La siguiente definición ¿utiliza recursión explícita? Justificar.

```
esHoja :: AB a -> Bool
esHoja Nil = False
esHoja (Bin i r d) = esNil i && esNil d
  where esNil Nil = True
        esNil _ = False
```

Respuesta: NO, porque no se define ninguna función que se llame así misma. (no hay llamado recursivo).

Pregunta 2: Indicar el tipo del esquema de recursión estructural foldABNV para el siguiente tipo:

```
data ABNV a = Hoja a | Uni a (ABNV a) | Bi (ABNV a) a (ABNV a)
```

Respuesta: ~~foldABNV :: (a -> b) -> (a -> b -> b) -> (b -> a -> b -> b) -> ABNV a -> b~~
foldABNV :: (a -> b) -> (a -> b -> b) -> (b -> a -> b -> b) -> ABNV a -> b

Pregunta 3: ¿Qué tipo de recursión utiliza la siguiente definición? (Indicar la más específica).

```
listasQueSuman :: Int -> [[Int]]
listasQueSuman 0 = [[]]
listasQueSuman n | n > 0 = [x : xs | x <- [1..n], xs <- listasQueSuman (n-x)]
```

- | | |
|----------------|------------------------|
| a. Estructural | c. Global |
| b. Primitiva | d. Iterativa/a la cola |

Respuesta: C

Pregunta 4: Indicar cuál de las siguientes divisiones en casos puede justificarse por un Lema de Generación.

- | | | |
|---|---|--|
| a. Hay 2 casos: | b. La lista xs puede ser: | c. El árbol binario t puede ser: |
| <ul style="list-style-type: none"> altura i ≥ altura d altura i < altura d | <ul style="list-style-type: none"> [] [x] x:y:ys | <ul style="list-style-type: none"> Nil Bin i r d |
| (Con i, d :: AB a). | (Con x, y :: a, ys :: [a]). | (Con r :: a, i y d :: AB a). |

Respuesta: C

Pregunta 5: Definir el predicado unario P que se podría usar para demostrar la siguiente propiedad por inducción estructural:

$\forall xs :: [a]. \forall ys :: [a]. \text{length} (\text{entrelazar } xs \text{ } ys) = \text{length } xs + \text{length } ys$

Respuesta: $P(xs) = \forall ys :: [a]. \text{length} (\text{entrelazar } xs \text{ } ys) = \text{length } xs + \text{length } ys$
Para demostrar la propiedad, se demuestra que vale $P(xs)$.
 $\forall xs :: [a]$

Pregunta 6: Si se quisiera demostrar que vale $\neg P \vee Q \vdash Q \vee (P \Rightarrow Q)$ por deducción natural, ¿cuál sería el error al aplicar el siguiente paso?

$$\frac{\neg P \vee Q \vdash Q}{\neg P \vee Q \vdash Q \vee (P \Rightarrow Q)} (V_{i1})$$

en el contexto.

Respuesta: El error es que pierdo información necesaria para poder demostrarlo, ya que con $\neg P \vee Q$ no tengo forma de probar que siempre vale Q .

Pregunta 7: Indicar a qué término reduce en un paso la siguiente expresión, y mediante qué regla(s):
if isZero(zero) then Pred(Succ(zero)) else zero

Respuesta: reduce a if true then Pred(Succ(zero)) else zero, por reglas E-IF y E-IsZero.

Pregunta 8: Sea la extensión del Cálculo Lambda con listas vista en la guía:

$$M ::= \dots \mid []_\tau \mid M :: M \mid \text{case } M \text{ of } \{ [] \rightsquigarrow M \mid h :: t \rightsquigarrow M \} \mid \text{foldr } M \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow M$$

Indicar el error en la siguiente regla de cómputo para foldr:

$$\frac{\text{foldr } V_1 :: V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \rightarrow N}{\text{foldr } V_1 :: V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N} \text{ (E-FOLDR-CONS)}$$

Respuesta: El término N puede tener apariciones libres de las variables h y r , por lo que al reducir el foldr $V_1 :: V_2$ es necesario ligarlas con la cabeza y el resultado de la recursión. La regla debería ser:

$$\text{foldr } V_1 :: V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \longrightarrow * \\ * N \{ h := V_1, r := \text{foldr } V_2 \text{ base } \rightsquigarrow M; \text{rec}(h, r) \rightsquigarrow N \}$$

sobre la cola.